

Admin Approval for STAFF - Complete Implementation

Current Situation Analysis

Based on your OpenAPI spec:

-  Signup endpoint already mentions "Doctor, Staff (public signup)"
-  You already have `is_approved` field in the system
-  Doctor approval workflow is already documented

Goal: Make staff also require admin approval (same as doctors)

Implementation Steps

Step 1: Update Signup Endpoint (Backend)

Your signup endpoint should handle BOTH doctors AND staff with approval requirement.

```
python
```

```
# app/api/routes/users.py
```

```
@router.post("/users/signup", response_model=UserPublic)
```

```
async def register_user(
```

```
    user_in: UserRegister,
```

```
    db: Session = Depends(get_db)
```

```
):
```

```
    """
```

```
 Access: Doctor, Staff (public signup)
```

User self-registration endpoint for creating new doctor or staff accounts.

BOTH require admin approval.

Workflow:

1. User submits signup (doctor OR staff)
2. Email verification link sent
3. After email verification, account pending admin approval
4. Admin reviews and approves
5. User can login once approved

```
    """
```

```
# Check if email exists
```

```
existing = db.query(User).filter(User.email == user_in.email).first()
```

```
if existing:
```

```
    raise HTTPException(
```

```
        status_code=400,
```

```
        detail="Email already registered"
```

```
    )
```

```
# Determine role - default to "doctor" if not specified
```

```
# Staff can be selected via a "role" field in signup form
```

```
role = user_in.role if hasattr(user_in, 'role') and user_in.role else "doctor"
```

```
# Create user with approval required
```

```
user = User(
```

```
    email=user_in.email,
```

```
    hashed_password=get_password_hash(user_in.password),
```

```
    full_name=user_in.full_name,
```

```
    phone=user_in.phone,
```

```
    role=role, # Can be "doctor" or "staff"
```

```
# Doctor-specific fields (only if doctor)
```

```
registration_number=user_in.registration_number if role == "doctor" else None,
```

```
specialization=user_in.specialization if role == "doctor" else None,
```

```
clinic_name=user_in.clinic_name if role == "doctor" else None,
```

```
clinic_address=user_in.clinic_address if role == "doctor" else None,
```

```
# Approval status - BOTH doctors and staff start here
```

```
is_verified=False, # Email not verified yet
```

```
is_approved=False, # Admin approval pending
```

```
is_active=False # Cannot login until approved
```

```
)
```

```
db.add(user)
db.commit()
db.refresh(user)

# Send verification email
send_verification_email(user.email, user.full_name)

# Notify admin about new signup (doctor OR staff)
notify_admin_new_user_signup(user)

return user
```

Step 2: Update UserRegister Schema (Add Role Field)

```
python
```

```
# app/models/user.py

from enum import Enum
from pydantic import BaseModel, EmailStr, Field

class UserRole(str, Enum):
    DOCTOR = "doctor"
    STAFF = "staff"
    ADMIN = "admin"

class UserRegister(BaseModel):
    """API INPUT MODEL for user registration (doctors and staff)"""

    # Required for all users
    email: EmailStr
    password: str = Field(min_length=8, max_length=128)
    full_name: str = Field(max_length=255)
    phone: str | None = Field(None, max_length=20)

    # NEW: Allow role selection (doctor or staff)
    role: UserRole = UserRole.DOCTOR # Default to doctor

    # Optional - for doctors only
    registration_number: str | None = Field(
        None,
        max_length=100,
        description="Medical license/registration number (required for doctors)"
    )
    specialization: str | None = Field(
        None,
        max_length=255,
        description="Medical specialization (for doctors)"
    )
    clinic_name: str | None = Field(
        None,
        max_length=255,
        description="Practice/clinic name (for doctors)"
    )
    clinic_address: str | None = Field(
        None,
        description="Practice/clinic address (for doctors)"
    )

```

Step 3: Update Login Endpoint (Check Approval for Both Roles)

```
python
```

```

@router.post("/login", response_model=LoginResponse)
async def login(
    credentials: LoginRequest,
    db: Session = Depends(get_db)
):
    """
    Login with email and password
    Checks approval status for BOTH doctors and staff
    """

    # Authenticate user
    user = authenticate_user(db, credentials.email, credentials.password)

    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect email or password"
        )

    # Check email verification
    if not user.is_verified:
        raise HTTPException(
            status_code=403,
            detail="Please verify your email first. Check your inbox for the verification link."
        )

    # Check approval status - APPLIES TO BOTH DOCTORS AND STAFF
    if user.role in ["doctor", "staff"] and not user.is_approved:
        raise HTTPException(
            status_code=403,
            detail=f"Your {user.role} account is pending admin approval. You will receive an email notification once approved (usually within 24 hours)"
        )

    # Check active status
    if not user.is_active:
        raise HTTPException(
            status_code=403,
            detail="Your account has been deactivated. Please contact support."
        )

    # Create access token
    access_token = create_access_token(
        data={"sub": str(user.id), "role": user.role, "entity": "user"}
    )

    # Update last login
    user.last_login = datetime.utcnow()
    db.commit()

    return LoginResponse(
        access_token=access_token,
        token_type="bearer",
    )

```

```
user=user
```

```
)
```

Step 4: Admin Endpoints (Unified for Both Roles)

```
python
```

```
@router.get("/admin/pending-users", response_model=UsersPublic)
async def get_pending_users(
    role: str | None = None, # Filter by role: "doctor", "staff", or None for all
    skip: int = 0,
    limit: int = 100,
    current_user: User = Depends(get_current_admin),
    db: Session = Depends(get_db)
):
    """
    🛡️ Admin endpoint to get all users pending approval

    Can filter by role:
    - role=doctor: Only pending doctors
    - role=staff: Only pending staff
    - role=None: All pending users (doctors + staff)
    """
    query = db.query(User).filter(
        User.is_verified == True, # Email verified
        User.is_approved == False # Approval pending
    )

    # Filter by role if specified
    if role:
        query = query.filter(User.role == role)
    else:
        # Get both doctors and staff (exclude admins)
        query = query.filter(User.role.in_(["doctor", "staff"]))

    # Get total count
    count = query.count()

    # Get paginated results
    users = query.offset(skip).limit(limit).all()

    return UsersPublic(data=users, count=count)
```

```
@router.post("/admin/approve-user/{user_id}", response_model=UserPublic)
async def approve_user(
    user_id: UUID,
    approve: bool = True, # True = approve, False = reject
    rejection_reason: str | None = None,
    current_user: User = Depends(get_current_admin),
    db: Session = Depends(get_db)
):
    """
    🛡️ Admin endpoint to approve or reject user signup
    """
```

Works for BOTH doctors and staff

Actions:

- approve=True: Activate the account

- approve=False: Reject with optional reason

"""

Get the user

user = db.query(User).filter(User.id == user_id).first()

if not user:

 raise HTTPException(status_code=404, detail="User not found")

if user.role not in ["doctor", "staff"]:

 raise HTTPException(
 status_code=400,
 detail="This endpoint is only for approving doctors and staff"
)

if approve:

 # APPROVE

 user.is_approved = True

 user.is_active = True

 user.approved_at = datetime.utcnow()

 user.approved_by = current_user.id

 db.commit()

 db.refresh(user)

 # Send approval email

 send_approval_email(user)

 # Log audit trail

 log_audit_action(
 db=db,
 user_id=user.id,
 action="user_approved",
 performed_by=current_user.id,
 details=f'{user.role.capitalize()} {user.full_name} approved"
)

 return user

else:

 # REJECT

 user.is_approved = False

 user.is_active = False

 user.rejection_reason = rejection_reason

 db.commit()

 db.refresh(user)

 # Send rejection email

 send_rejection_email(user, rejection_reason)

 # Log audit trail

 log_audit_action(
 db=db,
 user_id=user.id,
 action="user_rejected",
 performed_by=current_user.id,
 details=f'{user.role.capitalize()} {user.full_name} rejected'
)


```
db=db,  
user_id=user.id,  
action="user_rejected",  
performed_by=current_user.id,  
details=f'{user.role.capitalize()} {user.full_name} rejected: {rejection_reason}'  
)  
  
return user
```

Step 5: Email Notifications

python

```

# app/utils/email.py

def notify_admin_new_user_signup(user: User):
    """
    Notify admin when new user (doctor OR staff) signs up
    """
    admin_email = settings.ADMIN_EMAIL # e.g., "admin@clinic.com"

    # Role-specific subject
    subject = f"🔔 New {user.role.capitalize()} Signup: {user.full_name}"

    # Build email body based on role
    if user.role == "doctor":
        body = f"""
Hello Admin,

A new doctor has signed up and is waiting for approval:

👤 Name: {user.full_name}
✉ Email: {user.email}
📞 Phone: {user.phone or 'Not provided'}
🏥 Specialization: {user.specialization or 'Not provided'}
🪪 ID Registration #: {user.registration_number or 'Not provided'}
🏠 Clinic: {user.clinic_name or 'Not provided'}
📍 Address: {user.clinic_address or 'Not provided'}

📅 Signed up: {user.created_at}
✉ Email verified: {'Yes' if user.is_verified else 'Pending'}

To review and approve, visit your admin dashboard:
{settings.ADMIN_DASHBOARD_URL}/pending-users

---
Spine Homeo Admin Notification
"""
    else: # staff
        body = f"""
Hello Admin,

A new staff member has signed up and is waiting for approval:

👤 Name: {user.full_name}
✉ Email: {user.email}
📞 Phone: {user.phone or 'Not provided'}

📅 Signed up: {user.created_at}
✉ Email verified: {'Yes' if user.is_verified else 'Pending'}

To review and approve, visit your admin dashboard:
{settings.ADMIN_DASHBOARD_URL}/pending-users

---

```

Spine Homeo Admin Notification

"""

```
send_email(to=admin_email, subject=subject, body=body)
```

```
def send_approval_email(user: User):
```

"""

Send approval notification to user (doctor or staff)

"""

```
subject = f"👉 Your {user.role.capitalize()} Account Has Been Approved!"
```

```
body = f"""
```

```
Hi {user.full_name},
```

Great news! Your {user.role} account has been approved by our admin team.

You can now login and start using the Spine Homeo system:

🔗 [Login here: {settings.FRONTEND_URL}/login](#)

Your account details:

✉ **Email:** {user.email}

👤 **Role:** {user.role.capitalize()}

If you have any questions, feel free to contact our support team.

Welcome aboard!

Spine Homeo Team

"""

```
send_email(to=user.email, subject=subject, body=body)
```

```
def send_rejection_email(user: User, reason: str | None = None):
```

"""

Send rejection notification to user

"""

```
subject = f"Account Application Update"
```

```
reason_text = f"\n\nReason: {reason}" if reason else ""
```

```
body = f"""
```

```
Hi {user.full_name},
```

Thank you for your interest in joining Spine Homeo as a {user.role}.

Unfortunately, we are unable to approve your account at this time. {reason_text}

If you believe this is an error or would like to provide additional information, please contact our support team at {settings.SUPPORT_EMAIL}.

```
---
Spine Homeo Team

'''

send_email(to=user.email, subject=subject, body=body)
```

Frontend Changes

1. Signup Form - Add Role Selection

```
html
```

```
<!-- signup.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Sign Up - Spine Homeo</title>
  <style>
    .form-group { margin-bottom: 15px; }
    label { display: block; margin-bottom: 5px; font-weight: bold; }
    input, select, textarea {
      width: 100%;
      padding: 10px;
      border: 1px solid #ddd;
      border-radius: 4px;
    }
    .role-selector {
      display: flex;
      gap: 15px;
      margin-bottom: 20px;
    }
    .role-option {
      flex: 1;
      padding: 20px;
      border: 2px solid #ddd;
      border-radius: 8px;
      cursor: pointer;
      text-align: center;
    }
    .role-option.selected {
      border-color: #1976d2;
      background: #e3f2fd;
    }
    .doctor-fields { display: none; }
    .doctor-fields.show { display: block; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Create Your Account</h1>

    <form id="signup-form">
      <!-- Role Selection -->
      <div class="form-group">
        <label>I am registering as:</label>
        <div class="role-selector">
          <div class="role-option selected" data-role="doctor">
            <h3>👨 Doctor</h3>
            <p>Medical practitioner</p>
          </div>
          <div class="role-option" data-role="staff">
            <h3>👩 Staff</h3>
            <p>Administrative staff</p>
          </div>
        </div>
      </div>
    </form>
  </div>
</body>
</html>
```

```
</div>
<input type="hidden" id="role" name="role" value="doctor">
</div>

<!-- Common Fields -->
<div class="form-group">
  <label>Full Name *</label>
  <input type="text" name="full_name" required>
</div>

<div class="form-group">
  <label>Email *</label>
  <input type="email" name="email" required>
</div>

<div class="form-group">
  <label>Password *</label>
  <input type="password" name="password" minlength="8" required>
</div>

<div class="form-group">
  <label>Phone</label>
  <input type="tel" name="phone">
</div>

<!-- Doctor-Only Fields -->
<div class="doctor-fields show" id="doctor-fields">
  <div class="form-group">
    <label>Medical Registration Number *</label>
    <input type="text" name="registration_number" id="reg_number">
  </div>

  <div class="form-group">
    <label>Specialization *</label>
    <input type="text" name="specialization" id="specialization"
      placeholder="e.g., Cardiologist, Orthopedic">
  </div>

  <div class="form-group">
    <label>Clinic/Hospital Name</label>
    <input type="text" name="clinic_name">
  </div>

  <div class="form-group">
    <label>Clinic Address</label>
    <textarea name="clinic_address" rows="3"></textarea>
  </div>
</div>

<button type="submit">Sign Up</button>

<p class="note">
```

After signup, your account will be reviewed by our admin team.

You'll receive an email once approved (usually within 24 hours).

</p>

</form>

</div>

<script>

// Handle role selection

const roleOptions = document.querySelectorAll('.role-option');

const roleInput = document.getElementById('role');

const doctorFields = document.getElementById('doctor-fields');

const regNumber = document.getElementById('reg_number');

const specialization = document.getElementById('specialization');

roleOptions.forEach(option => {

option.addEventListener('click', function() {

// Update selected state

roleOptions.forEach(o => o.classList.remove('selected'));

this.classList.add('selected');

// Update role value

const role = this.dataset.role;

roleInput.value = role;

// Show/hide doctor fields

if (role === 'doctor') {

doctorFields.classList.add('show');

regNumber.required = true;

specialization.required = true;

} else {

doctorFields.classList.remove('show');

regNumber.required = false;

specialization.required = false;

}

});

});

// Handle form submission

document.getElementById('signup-form').addEventListener('submit', async (e) => {
e.preventDefault();

const formData = new FormData(e.target);

const data = {

email: formData.get('email'),

password: formData.get('password'),

full_name: formData.get('full_name'),

phone: formData.get('phone'),

role: formData.get('role'),

};

// Add doctor fields if role is doctor

if (data.role === 'doctor') {

```
data.registration_number = formData.get('registration_number');
data.specialization = formData.get('specialization');
data.clinic_name = formData.get('clinic_name');
data.clinic_address = formData.get('clinic_address');
}

try {
  const response = await fetch('http://localhost:8000/users/signup', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  });

  if (response.ok) {
    alert('Signup successful! Please check your email to verify your account.');
```

 window.location.href = '/login.html';

```
  } else {
    const error = await response.json();
    alert('Signup failed: ' + error.detail);
  }
} catch (error) {
  console.error('Error:', error);
  alert('Signup failed. Please try again.');
```

```

}
});
</script>
</body>
</html>
```

2. Admin Dashboard - Updated

html


```
<!-- admin/pending-users.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Pending User Approvals</title>
  <style>
    /* [Previous styles remain the same] */
    .filter-tabs {
      display: flex;
      gap: 10px;
      margin-bottom: 20px;
    }
    .tab {
      padding: 10px 20px;
      border: 2px solid #ddd;
      background: white;
      cursor: pointer;
      border-radius: 4px;
    }
    .tab.active {
      background: #1976d2;
      color: white;
      border-color: #1976d2;
    }
    .role-badge {
      display: inline-block;
      padding: 3px 8px;
      border-radius: 12px;
      font-size: 12px;
      font-weight: bold;
    }
    .role-doctor { background: #e3f2fd; color: #1976d2; }
    .role-staff { background: #fff3e0; color: #f57c00; }
  </style>
</head>
<body>
  <div class="container">
    <h1>👤 Pending User Approvals</h1>

    <div class="filter-tabs">
      <button class="tab active" data-filter="all">All Pending</button>
      <button class="tab" data-filter="doctor">Doctors Only</button>
      <button class="tab" data-filter="staff">Staff Only</button>
    </div>

    <div class="stats">
      <strong>Pending Approvals: <span id="count">0</span></strong>
    </div>

    <table id="users-table">
      <thead>
        <tr>
```

```

        <th>Role</th>
        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
        <th>Specialization/Reg #</th>
        <th>Clinic</th>
        <th>Joined</th>
        <th>Action</th>
    </tr>
</thead>
<tbody id="users-tbody">
</tbody>
</table>
</div>

<script>
    const API_BASE = 'http://localhost:8000';
    const token = localStorage.getItem('admin_token');
    let currentFilter = 'all';

    // Tab filtering
    document.querySelectorAll('.tab').forEach(tab => {
        tab.addEventListener('click', function() {
            document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
            this.classList.add('active');
            currentFilter = this.dataset.filter;
            loadPendingUsers();
        });
    });

    async function loadPendingUsers() {
        try {
            let url = `${API_BASE}/admin/pending-users`;
            if (currentFilter !== 'all') {
                url += `?role=${currentFilter}`;
            }

            const response = await fetch(url, {
                headers: { 'Authorization': `Bearer ${token}` }
            });

            if (!response.ok) throw new Error('Failed to fetch');

            const data = await response.json();
            displayUsers(data.data);
            document.getElementById('count').textContent = data.count;

        } catch (error) {
            console.error('Error:', error);
            alert('Failed to load pending users');
        }
    }
}

```

```

function displayUsers(users) {
  const tbody = document.getElementById('users-tbody');
  tbody.innerHTML = "";

  users.forEach(user => {
    const roleClass = `role-${user.role}`;
    const roleBadge = `<span class="role-badg` + roleClass + `">${user.role.toUpperCase()}</span>`;

    // Different info based on role
    let extraInfo = "";
    if (user.role === 'doctor') {
      extraInfo = `${user.specialization || 'N/A'}<br><small>${user.registration_number || 'N/A'}</small>`;
    } else {
      extraInfo = 'N/A';
    }

    const row = document.createElement('tr');
    row.innerHTML = `
      <td>${roleBadge}</td>
      <td><strong>${user.full_name}</strong></td>
      <td>${user.email}</td>
      <td>${user.phone || 'N/A'}</td>
      <td>${extraInfo}</td>
      <td>${user.clinic_name || 'N/A'}</td>
      <td>${new Date(user.join_date).toLocaleDateString()}</td>
      <td>
        <button class="btn activate"
          onclick="approveUser('${user.id}', true)">
          ✓ Approve
        </button>
        <button class="btn deactivate"
          onclick="approveUser('${user.id}', false)">
          ✗ Reject
        </button>
      </td>
    `;
    tbody.appendChild(row);
  });
}

async function approveUser(userId, approve) {
  const action = approve ? 'approve' : 'reject';
  let reason = null;

  if (!approve) {
    reason = prompt('Rejection reason (optional):');
  }

  if (!confirm(`Are you sure you want to ${action} this user?`)) {
    return;
  }
}

```

```

    try {
      let url = `${API_BASE}/admin/approve-user/${userId}?approve=${approve}`;
      if (reason) {
        url += `&rejection_reason=${encodeURIComponent(reason)}`;
      }

      const response = await fetch(url, {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${token}` }
      });

      if (response.ok) {
        alert(approve ? 'User approved!' : 'User rejected');
        loadPendingUsers();
      } else {
        alert('Action failed');
      }
    } catch (error) {
      console.error('Error:', error);
      alert('Action failed');
    }
  }

  // Load on page load
  loadPendingUsers();
</script>
</body>
</html>

```

Complete Flow for Staff

STAFF SIGNUP & APPROVAL FLOW

Step 1: STAFF SIGNUP

- └─ Staff member goes to signup page
 - └─ Selects "Staff" role (not doctor)
 - └─ Fills: Email, Password, Full Name, Phone
 - └─ Submits form
 - └─ Status: is_verified=FALSE, is_approved=FALSE, is_active=FALSE
- ↓ (Verification email sent)

Step 2: EMAIL VERIFICATION

- └─ Staff clicks link in email
- └─ Email is verified
- └─ Status: is_verified=TRUE, is_approved=FALSE, is_active=FALSE

⚠ Still cannot login (needs admin approval)

↓ (Admin notification email sent)

Step 3: ADMIN REVIEW

- Admin logs into dashboard
- Clicks "Staff Only" tab
- Sees: John Doe (Staff) - Pending
- Reviews info: Name, Email, Phone
- Clicks "Approve" button
- Status: is_verified=TRUE, is_approved=TRUE, is_active=TRUE

↓ (Approval email sent to staff)

Step 4: STAFF LOGIN

- Staff goes to login page
- Enters credentials
- All checks pass ✓
 - is_verified = TRUE ✓
 - is_approved = TRUE ✓
 - is_active = TRUE ✓
- Successfully logged in → Staff Dashboard

✅ Summary

What Changed:

- ✅ **UserRegister schema** - Added `role` field (doctor or staff)
- ✅ **Signup endpoint** - Handles both doctors and staff equally
- ✅ **Login endpoint** - Checks approval for both roles
- ✅ **Admin endpoints** - Unified approval system
- ✅ **Frontend** - Role selector in signup form
- ✅ **Admin dashboard** - Filter by role (all/doctors/staff)

Key Points:

- **BOTH** doctors and staff require admin approval
- **Same workflow** for both roles
- **Different fields** collected (doctors need license info, staff don't)
- **Role-specific** email notifications
- **Filterable** admin dashboard

Testing Checklist:

- ☐ Staff signup → Email verification → Pending approval
- ☐ Staff login before approval → 403 error with message
- ☐ Admin sees staff in pending list

- ☐ Admin approves staff → Success
- ☐ Staff login after approval → Success ✓
- ☐ Doctor workflow still works (not broken)

Implementation time: ~15 minutes (if you already have doctor approval working)